PROCEEDINGS A

royalsocietypublishing.org/journal/rspa

Research

Cite this article: Bucci MA, Semeraro O, Allauzen A, Wisniewski G, Cordier L, Mathelin L. 2019 Control of chaotic systems by deep reinforcement learning. *Proc. R. Soc. A* **475**: 20190351.
http://dx.doi.org/10.1098/rspa.2019.0351

Received: 4 June 2019
Accepted: 10 October 2019

Subject Areas:
artificial intelligence, fluid mechanics, mechanical engineering

Keywords:
deep reinforcement learning, Kuramoto–Sivashinsky, flow control

Author for correspondence:
M. A. Bucci
e-mail: michelealessandro.bucci@limsi.fr

Electronic supplementary material is available online at https://doi.org/10.6084/m9.figshare.c.4711487.

THE ROYAL SOCIETY PUBLISHING

Control of chaotic systems by deep reinforcement learning

M. A. Bucci[1], O. Semeraro[1], A. Allauzen[1], G. Wisniewski[1], L. Cordier[2] and L. Mathelin[1]

[1]LIMSI, CNRS, Université de Paris-Saclay, Orsay, France
[2]Institut Pprime, CNRS, Université de Poitiers, ISAE-ENSMA, Poitiers, France

MAB, 0000-0002-8048-7781

Deep reinforcement learning (DRL) is applied to control a nonlinear, chaotic system governed by the one-dimensional Kuramoto–Sivashinsky (KS) equation. DRL uses reinforcement learning principles for the determination of optimal control solutions and deep neural networks for approximating the value function and the control policy. Recent applications have shown that DRL may achieve superhuman performance in complex cognitive tasks. In this work, we show that using restricted localized actuation, partial knowledge of the state based on limited sensor measurements and model-free DRL controllers, it is possible to stabilize the dynamics of the KS system around its unstable fixed solutions, here considered as target states. The robustness of the controllers is tested by considering several trajectories in the phase space emanating from different initial conditions; we show that DRL is always capable of driving and stabilizing the dynamics around target states. The possibility of controlling the KS system in the chaotic regime by using a DRL strategy solely relying on local measurements suggests the extension of the application of RL methods to the control of more complex systems such as drag reduction in bluff-body wakes or the enhancement/diminution of turbulent mixing.

1. Introduction

With the availability of unprecedented computational resources, the maturity of modelling in many areas of engineering has yielded systems close to optimality in the context of the well-known settings they were engineered for. To further improve such systems, a scientific challenge lies in rendering these systems adaptive

by design to changes in the operating conditions. Enlarging the perspective from the engineering standpoint, current environmental needs have also invigorated the research effort on *flow control* applications. For example, carbon dioxide emissions are considered to be among the main causes of global warming and any reduction of these emissions could lead to an attenuation of this effect. Increasing the efficiency of existing technologies for the production of sustainable energy can lead to high potential benefits or to a substantial reduction in oil consumption in the transport economy sector. Not surprisingly, a rather large body of literature is already available on the many different efforts in this realm, with varying assumptions on the system, such as linearity of the governing equations, or on the amount of information one has on the system, such as whether or not a state vector is observable. The interested reader is referred to earlier reviews [1–3].

Active control for the optimization of the performance can be introduced via adequate strategies capable of modifying the system response in a prescribed manner (*open loop*) or as a function of some observations of the system at hand (*closed loop*). In both cases, the challenge is to infer an efficient and robust control strategy, hereafter termed *policy*, improving upon the retained objective function. In the usual *model-based* approach, a dynamical model is used to describe the behaviour of the system. This model allows us to predict the effect of a given control action and, therefore, can be used to derive the best control strategy for optimal performance. However, a physical model[1] is not always available. In addition to the systems for which the governing equations are simply unknown or very poorly known, there are many situations where solving the governing equations is too slow with respect to the dynamics at play to be useful. While reduced-order models may help in solving an approximate system meeting real-time constraints, they usually lack robustness and can critically lose accuracy when control is applied, resulting in poor performance at best.

A different line of control strategy relies on a *data-driven* approach. In this view, no model is assumed to be known and the control command is derived based on past observations only. In a training step, control actions are applied to the system and observations are made, possibly including the evaluation of the objective function. From this collection of observations, an input–output (I/O) model is built and subsequently used to control the system. Typical of this viewpoint are extremum-seeking [4], control strategies that rely on system-identification techniques that lead to auto-regressive models, such as ARMAX models, subspace methods or realization-type identification algorithms like ERA (see applications in [5–7] or the monographs in [8,9]). Recently, so-called machine learning control was proposed, which mainly relies on genetic programming [10], a symbolic regression algorithm. All these techniques are more directly compatible with real systems since they do not require a physical model or prohibitive computational power in the control step. Moreover, they do not assume knowledge of a full state vector and can deal with partial observations by leveraging the practical necessity of relying on a limited set of sensors and fast computations.

In the present work, we follow this rationale by considering a reinforcement learning (RL) strategy [11] for the closed-loop nonlinear control. RL is a well-established technique mainly originating from the robotics community and has gained widespread popularity in some recent media applications, achieving superhuman performances in the `go` and `shogi` games as well as self-teaching for the `StarCraft II` videogame [12], or in revenue management [13], to cite only a few examples. As a data-driven technique, it shares the applicability of other I/O approaches, e.g. no expensive computational model to simulate and the ability to use only limited sensors, in contrast to a full state vector. With respect to the aforementioned techniques, RL algorithms are characterized by better properties of convergence towards the minima of the cost functional and can potentially lead to more efficient learning [14]. These advantages are mainly due to the theoretical foundation on which RL is grounded. Indeed, RL dates back to the 1950s, when the problem of optimal control was solved by Richard Bellman through the introduction of dynamic

[1]The term *model* is ambiguous. Here, the term *model* refers to a *physical model*. This meaning differs from its use in the machine learning community where the term is more related to a parametrized function that links inputs to outputs.

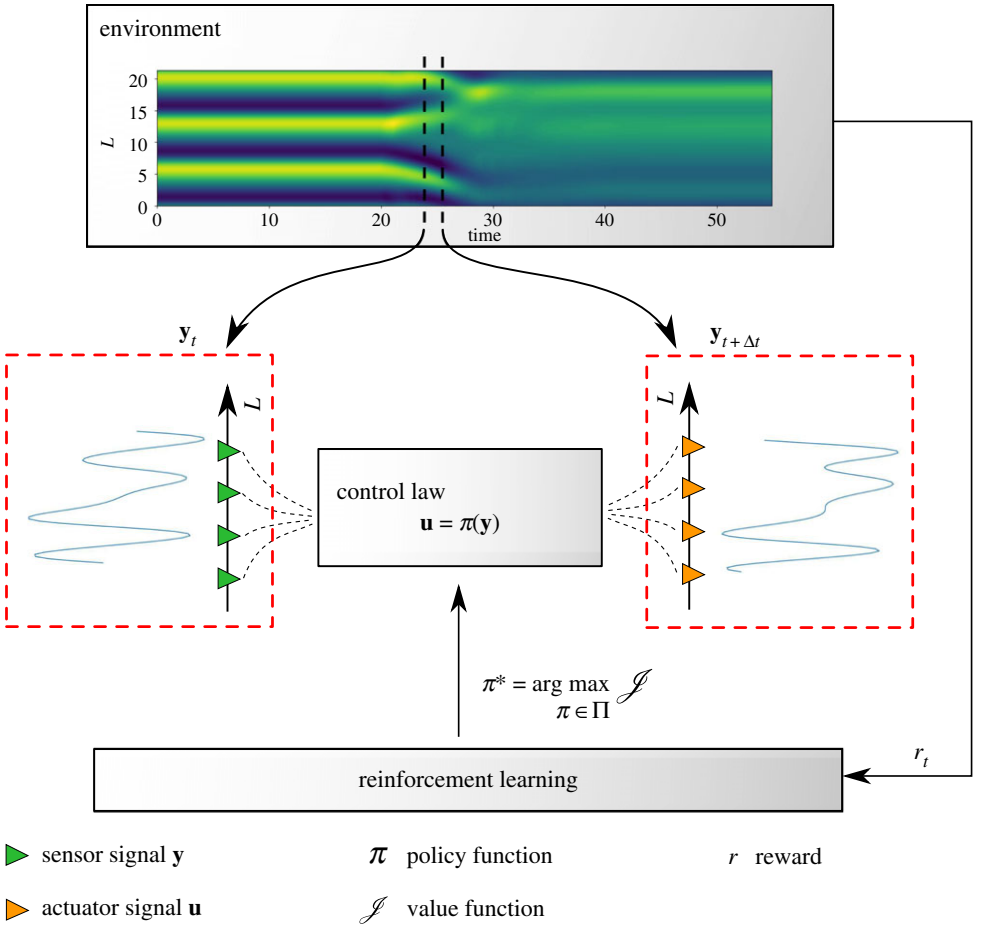

Figure 1. Overview of the reinforcement learning (RL) strategy for the Kuramoto–Sivashinsky equation considered in §4. In the RL framework, an *agent* interacts with an *environment* by making *observations* $\mathbf{y}$ and performing *actions* $\mathbf{u}$. In return, the agent receives a *reward* that depends directly on the changes of the environment induced by the action. The objective of RL is to determine the action $\mathbf{u}$ to impose on the environment in order to maximize a given *value function* $\mathscr{J}$ that represents the cumulative rewards over time. The action $\mathbf{u}$ is determined via the *policy function* $\boldsymbol{\pi}$, which maps measurements $\mathbf{y}$ taken from the environment to the action space. In our case, the measurements are taken by localized sensors (indicated by green triangles in the figure), while the action $\mathbf{u}$ feeds the actuators (indicated by orange triangles); the dynamics of the system is affected by the actuators' action such that the value function is maximized. In §4, we consider eight sensor measurements and four actuators. (Online version in colour.)

programming [15], leading—in the continuous formulation—to the Hamilton–Jacobi–Bellman (HJB) equation.

Figure 1 provides a sketch of the RL methodology for the control of a dynamical system. The physical system under consideration, termed the *environment* in the RL literature, is observed at time t by a so-called *agent* through a set of localized measurements $\mathbf{y}(t) \in \mathbb{R}^n$. The agent performs an action $\mathbf{u}(t)$ that changes the future state of the environment and, in return, receives a *reward* that represents the degree at which a state/action pair is desirable for the targeted objective function. This measure of performance to be optimized is then defined as the expectation of the (discounted) cumulative rewards over time. The action $\mathbf{u}(t) \in \mathbb{R}^m$ is evaluated from the current observations via the *control policy* $\boldsymbol{\pi}$, defined as $\mathbf{u}(t) = \boldsymbol{\pi}(\mathbf{y}(t))$, which represents a mapping from the observation space ($\mathbb{R}^n$) to the action space ($\mathbb{R}^m$). The policy is defined in a given class Π of multivariate functions. The *value function* $\mathscr{J} : \mathbb{R}^n \to \mathbb{R}$ or cost-to-go function of a policy $\boldsymbol{\pi}$ over a

time horizon gives the cumulative rewards when $\mathbf{y}_t$ is the current measurement and the system follows policy $\boldsymbol{\pi}$ thereafter. The optimal policy, denoted $\boldsymbol{\pi}^\star$, maximizes $\mathscr{J}$ over Π.

Applications of RL have been mostly restricted to the discrete settings, where the number of possible actions is potentially huge but finite [16]. A notable extension to the continuous framework includes the work by Gorodetsky and collaborators using function trains [17]. Our earlier efforts in bringing RL to the context of fluid flows involved Q-learning [18] and temporal difference-based continuous approaches [19,20]. Recent efforts from the literature involve optimizing a collective swimming strategy [21] and the control of flow around a circular cylinder in a laminar regime [22], among others. Here, we demonstrate the performance of our methodology in terms of the quality of the control strategy (achieved performance) and robustness with respect to perturbations to the system and sensor measurements. In this proof-of-concept work, we focus on the control of a nonlinear, chaotic dynamical system, the one-dimensional Kuramoto–Sivashinsky (KS) equation. The system is described in a space–time domain by a fourth-order partial differential equation (PDE). The KS system exhibits a wide range of dynamics, from the steady case to chaotic regimes depending on the extent of the spatial domain [23,24]. Similar to Navier–Stokes systems when the dissipation is high and the spatial domain small (e.g. a minimal flow unit in channel flows at low to moderate Reynolds numbers [25]), the KS system exhibits a low-dimensional dynamical behaviour, lying in a finite-dimensional space associated with non-trivial invariant solutions. When larger domains are considered, the dimension of the KS system increases, as well as the number of positive Lyapunov exponents, and the dynamics of the system can become spatiotemporally chaotic or weakly turbulent [26]. In that sense, the complexity of the KS dynamics, combined with the low computational costs of its numerical solution, makes this system an ideal starting point for the analysis of an RL controller, before tackling more challenging Navier–Stokes systems, such as bluff-body wakes or turbulent boundary layers.

The paper is organized as follows. The basics of our control strategies are presented in §§2 and 3. In §2, we review how RL is connected to the classical optimal control problem. We first derive the HJB equation from optimality principles, and next particularize in the discrete time settings to obtain the Bellman equation. The connection with linear optimal controllers is further described in appendix A. In §3a, we briefly discuss the different classes of RL algorithms. In §3b, the deep deterministic policy gradient (DDPG) RL technique we are using is then presented. The DRL methodology is illustrated with the control of the KS model in §4. The paper summarizes conclusions in §5.

2. From nonlinear optimal control to reinforcement learning

This section briefly introduces the mathematical background of RL. For a deeper introduction to RL, we refer the interested reader to [11,27]. From the methodological viewpoint, we introduce RL following the approach taken in [28,29], starting from the definition of optimal control (§3a) as the basis for the HJB equation (§3b) and stressing the analogies with optimal control theory. Further, we show how we can derive the time-discrete counterpart of the HJB equation, the Bellman equation (§3c), which serves as the theoretical and mathematical ground for RL applications.

(a) Definition of the control problem

We consider a dynamical system equipped with localized inputs (*actuators*) and outputs (*sensors*); these elements are organized along the stream-wise coordinate x in figure 1. Hereafter, the combination of the system we aim at controlling, the actuators and the sensors, will be simply referred to as the *plant*, following the classical terminology in control theory. From the mathematical viewpoint, this corresponds to defining the state-space model that,

in the general case, reads

$$\frac{\mathrm{d}\mathbf{v}}{\mathrm{d}t} = \mathbf{f}(\mathbf{v}(t), \mathbf{u}(t), t) \tag{2.1a}$$

and

$$\mathbf{y}(t) = \mathbf{g}(\mathbf{v}(t)). \tag{2.1b}$$

Equation (2.1*a*) is the state equation, where the map $\mathbf{f}$ propagates in time the state $\mathbf{v} \in \mathbb{R}^N$. In the output equation (2.1*b*), the map $\mathbf{g}$ is a function which associates the observed state $\mathbf{v}$ with a vector at time t. In the following, the observable $\mathbf{y}(t)$ will be referred to as the sensor signal. In the optimal control problem [30], we aim at defining a control signal $\mathbf{u} \in \mathbb{R}^m$ feeding the actuators, based on the sensor measurements $\mathbf{y} \in \mathbb{R}^n$, such that an objective function $\mathscr{J}$ is minimized

$$\mathscr{J} = h\left(\mathbf{v}(T), T\right) + \int_0^T r\left(\mathbf{v}(\tau), \mathbf{u}(\tau), \tau\right)\, \mathrm{d}\tau, \tag{2.2}$$

where h is a specified function, T is the optimization horizon and r is the reward associated with the state $\mathbf{v}$ and the action $\mathbf{u}$. According to [28], this optimization problem can be embedded in a larger class of problems by considering

$$\mathscr{J}\left(\mathbf{v}_t, t, \underset{t\le\tau\le T}{\mathbf{u}(\tau)}\right) = h(\mathbf{v}(T), T) + \int_t^T r(\mathbf{v}(\tau), \mathbf{u}(\tau), \tau)\, \mathrm{d}\tau, \tag{2.3}$$

where t can be any value less than or equal to T. Following the convention tacitly introduced in §1, we note that $\mathscr{J}^\star = \max_{\boldsymbol{\pi}} \mathscr{J}$, the optimal value of the objective function. The controller (or more properly the *compensator*) provides the mapping between the measurements $\mathbf{y}$ of the system and the control actions $\mathbf{u}$. For now, we stress that the aim of RL is to determine an optimal policy function $\boldsymbol{\pi}^\star$ that describes the optimal control $\mathbf{u}^\star$ from the current observations $\mathbf{y}$ [31,32] following

$$\mathbf{u}^\star(t) = \boldsymbol{\pi}^\star(\mathbf{y}(t), t). \tag{2.4}$$

(b) Hamilton–Jacobi–Bellman equation

The optimal control problem stated in (2.1) and (2.2) can be solved by maximizing an augmented Lagrangian [30], where the governing equations act as constraints. After optimal conditions are imposed on the Lagrangian, a direct-adjoint optimality system can be derived. When the function $\mathbf{f}$ is linear or can be linearized, and the objective function is quadratic, the final controller is obtained as the solution to a Riccati equation (see appendix A for the derivation). Here, we focus on the general nonlinear case (2.2) and follow a dynamic programming approach to solve the optimal control problem. The objective is to determine a PDE for the optimal objective function (or value function in the RL terminology) $\mathscr{J}^\star$ such that the corresponding action satisfies the constraint given by the state equation (2.1). By definition, the maximum value of the objective function is equal to

$$\begin{aligned}\mathscr{J}^\star(\mathbf{v}(t), t) &= \max_{\boldsymbol{\pi}} \mathscr{J}\left(\mathbf{v}_t, t, \underset{t\le\tau\le T}{\mathbf{u}(\tau)}\right)\\ &= \max_{\substack{\boldsymbol{\pi}(\tau)\\ t\le\tau\le T}} \left[\int_t^T r(\mathbf{v}(\tau), \mathbf{u}(\tau), \tau)\, \mathrm{d}\tau + h(\mathbf{v}(T), T)\right].\end{aligned} \tag{2.5}$$

By splitting the integrand in (2.5) between the immediate reward in the interval $[t,\ t + \Delta t]$ and the future value function at $t + \Delta t$, we obtain

$$\mathscr{J}^\star(\mathbf{v}(t), t) = \max_{\substack{\boldsymbol{\pi}(\tau)\\ t\le\tau\le T}} \left[\int_t^{t+\Delta t} r\, \mathrm{d}\tau + \int_{t+\Delta t}^T r\, \mathrm{d}\tau + h(\mathbf{v}(T), T)\right]. \tag{2.6}$$

The principle of optimality requires that

$$\mathscr{J}^\star(\mathbf{v}(t),t) = \max_{\substack{\boldsymbol{\pi}(\tau)\\ t\le\tau\le t+\Delta t}} \left[\int_t^{t+\Delta t} r\,\mathrm{d}\tau + \mathscr{J}^\star(\mathbf{v}(t+\Delta t), t+\Delta t)\right]. \tag{2.7}$$

Expanding $\mathscr{J}^\star(\mathbf{v}(t+\Delta t), t+\Delta t)$ in a Taylor series about $(\mathbf{v}(t),t)$ and taking the limit as $\Delta t \to 0$ gives

$$-\dot{\mathscr{J}}^\star(\mathbf{v}(t),t) = \max_{\boldsymbol{\pi}(t)}[r(\mathbf{v}(t),\mathbf{u}(t),t) + \mathscr{J}^\star_{\mathbf{v}}(\mathbf{v}(t),t)\mathbf{f}(\mathbf{v}(t),\mathbf{u}(t),t)], \tag{2.8}$$

where $\dot{\mathscr{J}}^\star$ is the temporal derivative of the optimal value function and $\mathscr{J}^\star_{\mathbf{v}}$ is the derivative with respect to the state. Equation (2.8) is the HJB equation. This equation is continuous in time and defined backward. The terminal condition is given by

$$\mathscr{J}^\star(\mathbf{v}(T),T) = h(\mathbf{v}(T),T). \tag{2.9}$$

The HJB equation is a sufficient condition for an optimum [33,34]. Indeed, a value function might fail to satisfy the differentiability and continuity conditions that are required to solve (2.8) and yet still be optimal. When solved over the whole state space and when the value function is continuously differentiable, the HJB equation becomes a necessary and sufficient condition for an optimum [28]. In the case of a continuous state–action space, the optimal policy $\boldsymbol{\pi}^\star$ is the one that produces the optimal trajectory by obeying the HJB equation. However, the whole state–action space is rarely known, especially when the dimension of $\mathbf{f}$ is large.

Note that, for infinite horizon optimizations, it is common to introduce a *discount rate factor*, $\rho > 0$, that penalizes the immediate reward in the future. In this case, (2.5) becomes

$$\mathscr{J}^\star(\mathbf{v}(t),t) = \max_{\boldsymbol{\pi}} \int_t^\infty \mathrm{e}^{-\rho\tau} r(\mathbf{v}(\tau),\mathbf{u}(\tau),\tau)\,\mathrm{d}\tau. \tag{2.10}$$

If we assume that $\mathscr{J}^\star$, $\mathbf{f}$ and r do not explicitly depend on time t, the HJB equation then finally can be rewritten as

$$\rho\,\mathscr{J}^\star(\mathbf{v}) = \max_{\boldsymbol{\pi}}[r(\mathbf{v},\mathbf{u}) + \mathscr{J}^\star_{\mathbf{v}}(\mathbf{v})\mathbf{f}(\mathbf{v},\mathbf{u})]. \tag{2.11}$$

(c) Bellman equation

In the derivation of the HJB equation, we have tacitly assumed that the model (2.1) is known or can be inferred, for instance by system identification. This is usually not the case as, most of the time, only the state $\mathbf{v}$ or a reduced-order representation at a given time t can be accessed, for instance by instantaneous measurements. In this case, the right framework is to consider a Markov decision process (MDP) for which the decision making is formulated by means of a transition matrix expressing the probability of evolving from one state to another under the chosen action $\mathbf{u}$. This framework introduces a discrete time stochastic control process and, as such, requires the reformulation of the objective function in terms of expectation [11]. Letting $\mathbf{v}_t$ and $\mathbf{u}_t$ be the state and the action at discrete time t, respectively, and $\mathbf{v}_{t+\Delta t}$ be the state at $t+\Delta t$, (2.11) can be rewritten as

$$\mathcal{J}^\star(\mathbf{v}_t) = \max_{\mathbf{u}}\left[r(\mathbf{v}_t,\mathbf{u}_t)\Delta t + \gamma\,\mathcal{J}^\star(\mathbf{v}_{t+\Delta t})\right], \tag{2.12}$$

where $\gamma = \exp(-\Delta t\,\rho)$ is the discount factor. Including Δt in the definition of $r(\mathbf{v}_t,\mathbf{u}_t)$, (2.12) is the Bellman optimality equation [15]. This equation, which describes the evolution of the optimal value function under the optimal policy $\boldsymbol{\pi}^\star$, is the foundation of the dynamic programming theory.

Let $\mathscr{J}^{\boldsymbol{\pi}}(\mathbf{v}_t)$ denote the long-term reward achieved for the state $\mathbf{v}_t$, and following a particular policy π. With the developments made in §2b for the HJB equation, we derive the Bellman

equation for the value function given by

$$\mathscr{J}^{\boldsymbol{\pi}}(\mathbf{v}_t) = r(\mathbf{v}_t, \mathbf{u}_t) + \gamma\, \mathscr{J}^{\boldsymbol{\pi}}(\mathbf{v}_{t+\Delta t}). \tag{2.13}$$

Similarly, we can derive a Bellman equation for the *state–action value function* $Q^{\boldsymbol{\pi}}(\mathbf{v}_t, \mathbf{u}_t)$ or *Q-function*. This quantity is a measure of the long-term reward assuming the agent is in state $\mathbf{v}_t$, performs action $\mathbf{u}_t$ and then continues following some policy π. The Bellman equation for the Q-function is

$$Q^{\pi}(\mathbf{v}_t, \mathbf{u}_t) = r(\mathbf{v}_t, \mathbf{u}_t) + \gamma Q^{\pi}(\mathbf{v}_{t+\Delta t}, \mathbf{u}_{t+\Delta t}). \tag{2.14}$$

At this point, a few remarks are in order, as follows.

(i) Since $\rho > 0$ and $\Delta t > 0$, the discount factor $\gamma \in (0, 1)$. In the MDP framework, the value function can be represented as the cumulative discounted reward or return R_t defined by

$$\mathscr{J}^{\boldsymbol{\pi}}(\mathbf{v}_t) = R_t = \sum_{l=0}^{\infty} \gamma^l r(\mathbf{v}_{t+l\Delta t}). \tag{2.15}$$

Note that the return is often denoted as G_t in the specialized literature. The two benefits of introducing a discounted reward are that the return is well defined for infinite series ($l \to \infty$) and that it gives a greater weight to earlier rewards, meaning that we care more about imminent rewards and less about rewards we will receive in the future.

(ii) Equation (2.12) highlights how the discounted infinite-horizon optimal problem can be decomposed into a series of local optimal problems. This is *Bellman's principle of optimality* [35]: an optimal policy has the property that, whatever the initial state and initial decision, the remaining decision must constitute an optimal policy with regard to the state resulting from the first decision.

(iii) In (2.13), the model (2.1) does not appear explicitly. If $\mathbf{v}_t$ can be observed, and the reward r can be measured, it is possible to recover $\mathscr{J}^{\pi}(\mathbf{v}_t)$ by inference from the agent–environment interaction, ensuring that $\mathscr{J}^{\pi}(\mathbf{v}_t)$ is the solution of the Bellman equation (2.13). This is precisely what the *reinforcement learning* (RL) approach does. In this framework, the policy and the value function are deterministic or stochastic functions. These functions can be represented as tables when the number of states and actions are sufficiently low. When there are more state and action variables, the *curse of dimensionality* occurs [36] and there is a need to approximate these functions as parametrized functional forms. Then, the *learning* process consists in optimizing the parameters so that the value function is maximized and the constraint imposed by the Bellman optimality (2.12) is satisfied. In *deep reinforcement learning* (DRL), the functions are represented by a neural network (NN).

Within the RL framework, the Bellman equation is formulated in terms of the expectation of the value function. In the value Bellman equation (2.13), the value function is replaced by the expected value ($\mathbb{E}[\mathscr{J}^{\boldsymbol{\pi}}(\mathbf{v}_t)]$). Similarly, in the cost–value Bellman equation (2.14), the Q-function is replaced by $\mathbb{E}[Q^{\pi}(\mathbf{v}_t, \mathbf{u}_t)]$.

3. Reinforcement learning

As mentioned above, the aim of RL is to find the optimal policy $\boldsymbol{\pi}^{\star}$ that maximizes the return (2.15). A crucial aspect for our application is the possibility for RL algorithms of learning directly from the observations resulting from the interactions of the agent with the environment. In that sense, we do not rely on the full knowledge of the state, but on a few localized measurements. Hereafter, the value function, the policy and the whole formulation will be given as parametrized functions of the observable $\mathbf{y}_t$. In §4, point-wise measurements will be used for the application on the KS system. In the following, we introduce a possible classification of the RL algorithms and specify our choices.

(a) Several classes of reinforcement learning algorithms

In the literature, many RL algorithms exist. In practice, the choice of the right algorithm often depends on the type of available actuators and sensors, on the system to control and on the task to fulfil. A general classification that embeds all RL algorithms can be made. Three classes of algorithms can be identified: (i) *actor only*, (ii) *critic only*, and (iii) *actor critic*, where the words *actor* and *critic* are synonyms for the policy and value function, respectively.

Actor-only methods work with a parametrized family of policies. The gradient of the value function, with respect to the parameters, is estimated, and the parameters are updated in the direction of improvement. In the DRL approach, the policy is represented by an NN to be inferred such that $\mathbf{u} = \boldsymbol{\pi}(\mathbf{y} \,|\, \boldsymbol{\omega})$, where $\boldsymbol{\omega}$ are the parameters of the NN. The learning process is guaranteed by choosing $\boldsymbol{\omega}$ such that the discounted reward R_t is maximized. In such a procedure, a single policy is evaluated by recording the system for a long time. This evaluation allows the computation of R_t and of the gradient of the value function with respect to the NN parameters of the policy, i.e. $\nabla_{\boldsymbol{\omega}} \mathscr{J}^{\boldsymbol{\pi}}(\mathbf{y}_t)$. The algorithms belonging to this class are referred to as `REINFORCE` algorithms [11]. A gradient-based optimization of the policy representation is carried out with stochastic gradient descent (SGD) algorithms (or closely related SGD-like strategies; for a recent review see [37]).

Critic-only methods rely exclusively on value function approximation and aim at learning an approximate solution to the Bellman equation, which will then hopefully prescribe a near-optimal policy. In the 'deep' flavour of the algorithms, the state–action value function $Q^{\boldsymbol{\pi}}$ is approximated by an NN, leading to $Q^{\boldsymbol{\pi}}(\mathbf{y}_t, \mathbf{u}_t \,|\, \boldsymbol{\theta})$, with $\boldsymbol{\theta}$ being the parameter vector of the NN. The parameters $\boldsymbol{\theta}$ are chosen such that the function $Q^{\boldsymbol{\pi}}$ is a solution of the Bellman equation (2.14). A Q-learning algorithm [38] may be used to approximate the optimal action–value function. The core of the algorithm is to iteratively update the function by using a weighted average of the old value and the new information:

$$Q^{\boldsymbol{\pi}}(\mathbf{y}_t, \mathbf{u}_t \,|\, \boldsymbol{\theta}) \longleftarrow Q^{\boldsymbol{\pi}}(\mathbf{y}_t, \mathbf{u}_t \,|\, \boldsymbol{\theta}) + \alpha \left(r(\mathbf{y}_t, \mathbf{u}_t) + \gamma \max_{\mathbf{u}} Q^{\boldsymbol{\pi}}(\mathbf{y}_{t+\Delta t}, \mathbf{u} \,|\, \boldsymbol{\theta}) - Q^{\boldsymbol{\pi}}(\mathbf{y}_t, \mathbf{u}_t \,|\, \boldsymbol{\theta}) \right), \tag{3.1}$$

where α is the learning rate ($0 < \alpha \leq 1$).

The use of NN in the Q-learning procedure establishes the deep Q-networks (DQNs) [39]. This class of algorithms is often referred to in the specialized literature as the first *'human-level'* control algorithm. The optimal policy does not need a function representation as it shall consist of choosing the action $\mathbf{u}$ that maximizes the optimal state–action value. For a finite number of possible actions, the Q-function is computed for each of these and the action that ensures the maximum is then chosen. The use of NNs imposes the implementation of some strategies to regularize the learning process, such as memory, prioritized experience, target NN, etc. [39].

From a mathematical point of view, the difference between the actor-only and critic-only approaches relies on the difference between Pontryagin's maximum principle and the Bellman principle. The first principle is based on a *perturbative* approach: the neighbourhood of the controlled trajectory is explored by perturbing the states in order to further minimize the cost function (curve optimization). The second principle is based on the solution of the HJB equation under the assumption of Markovianity of the system (function optimization), as explained in previous paragraphs. While Pontryagin's maximum principle is a necessary condition for optimality, fulfilment of the HJB equation is a necessary and sufficient condition if and only if the state–action is fully known (Legendre–Clebsch condition; see [14]).

The last class of algorithms, the actor–critic ones, combines the advantages of the two aforementioned approaches. The critic uses an approximation architecture to learn a value function, which is then used to update the actor's policy parameters in a direction of performance improvement. In DRL, two NNs are employed simultaneously, one for the policy and a second one for the value function. In that sense, a possible implementation consists in combining the `REINFORCE` and the Q-learning procedures such that the optimal controller is obtained [40]. From the algorithmic viewpoint, the coupling of the two NNs is due to the update of the networks that

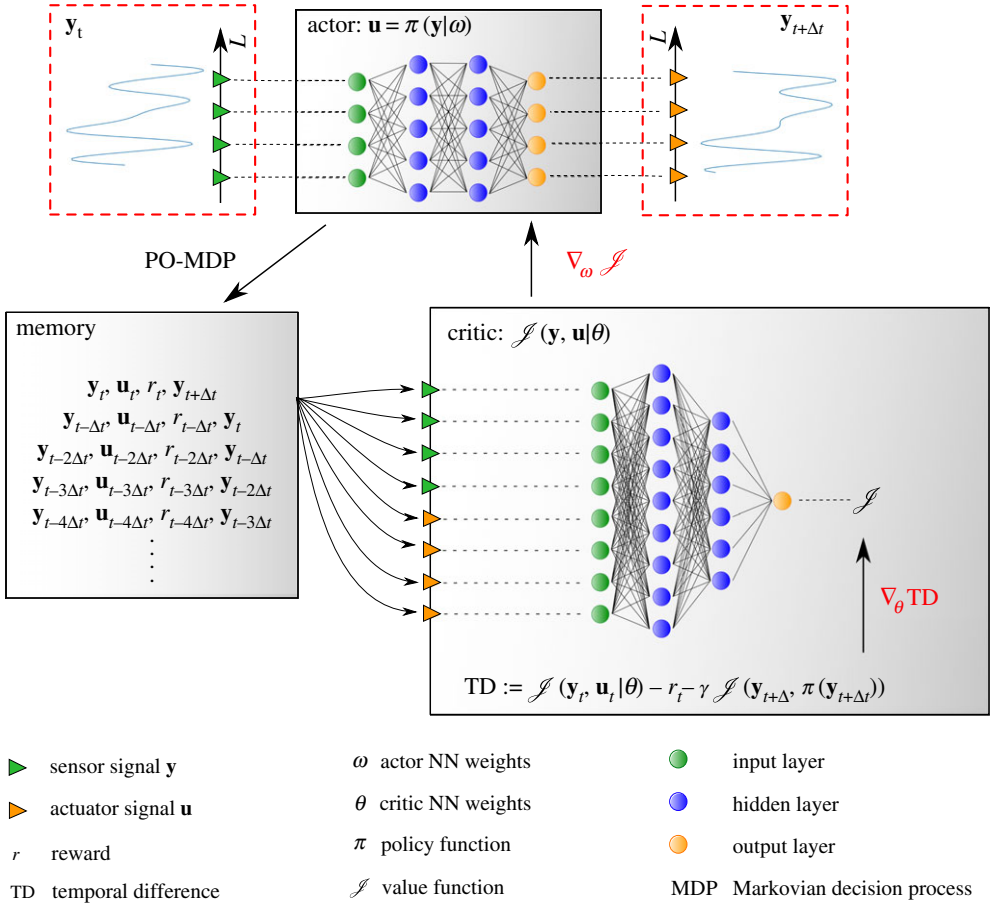

Figure 2. Graphical sketch for the DDPG algorithm. The strategy is an actor–critic controller, where each of the parts is implemented by means of an NN, as depicted in the sketch. (Online version in colour.)

is performed simultaneously by sharing the same expectation of the discounted reward. With a parametrized policy function, the limitation of the application of the critic-only approach to a discrete action space is circumvented.

(b) Deep deterministic policy gradient as an actor–critic algorithm for reinforcement learning

In this work, we consider the DDPG algorithm [41] as an actor–critic, model-free algorithm (see the graphical summary in figure 2). The advantage of DDPG compared with the DQN considered in [39] is that it can handle the continuous action domains that are often encountered in physical control tasks. Essentially, the DDPG algorithm adapts the theoretically grounded deterministic policy gradient (DPG) algorithm introduced in [42] to the DRL setting where NNs are employed to represent the policy and value functions. The basic idea of DPG is to update the policy parameter $\boldsymbol{\omega}$ in the direction of the gradient of $Q^{\boldsymbol{\pi}}$, rather than globally maximizing $Q^{\boldsymbol{\pi}}$, as is done classically in policy gradient methods. After application of the chain rule to $Q^{\boldsymbol{\pi}}(\mathbf{y}_t, \boldsymbol{\pi}(\mathbf{y}_t \mid \boldsymbol{\omega}))$, we can show that the policy improvement at each iteration number k can be decomposed into the gradient of the state–action value with respect to actions, and the gradient of the policy with

respect to the policy parameters (see eqns 6 and 7 on page 3 of [42], where the policy $\boldsymbol{\pi}$ is indicated as μ). We finally obtain

$$\boldsymbol{\omega}^{k+1} = \boldsymbol{\omega}^{k} + \alpha \mathbb{E}[\nabla_{\boldsymbol{\omega}} \boldsymbol{\pi}(\mathbf{y}_t \,|\, \boldsymbol{\omega}) \nabla_{\mathbf{u}_t} Q^{\boldsymbol{\pi}^k}(\mathbf{y}_t, \mathbf{u}_t)|_{\mathbf{u}_t = \boldsymbol{\pi}(\mathbf{y}_t \,|\, \boldsymbol{\omega})}], \tag{3.2}$$

where α is a learning rate.

The tuple corresponding to one time discrete transition $\{\mathbf{y}_t, \mathbf{u}_t, r_t, \mathbf{y}_{t+\Delta t}\}$ defines an MDP or partially observable Markov decision process (PO-MDP) in case only a partial measurement of the state is accessible. At each iteration, the PO-MDP is stacked in memory and successively used by the critic optimizer to reduce the temporal difference error, which is defined as

$$Q^{\boldsymbol{\pi}}(\mathbf{y}_t, \mathbf{u}_t \,|\, \boldsymbol{\theta}) - r(\mathbf{y}_t, \mathbf{u}_t) - \gamma Q^{\boldsymbol{\pi}}(\mathbf{y}_{t+\Delta t}, \mathbf{u}_{t+\Delta t} \,|\, \boldsymbol{\theta}). \tag{3.3}$$

Regardless of the optimized critic NN, the actor NN is also optimized according to (3.2).

The optimality of the control is guaranteed by the Bellman principle under the hypothesis that the state–action space is known. For this reason, the state–action space needs to be explored. The exploration is carried out by perturbing the parameters of the policy as

$$\mathbf{u}_t = \boldsymbol{\pi}(\mathbf{y}_t \,|\, \boldsymbol{\omega} + \mathcal{N}_1) + \mathcal{N}_2, \tag{3.4}$$

where $\mathcal{N}_1$ and $\mathcal{N}_2$ are two noise processes. $\mathcal{N}_1$ is essential at the beginning of the exploration process to introduce unbiased and uncorrelated random actions. The NN is a strongly nonlinear representation of the true optimal policy function; a small variation of the parameters can thus lead to drastic changes in the output action. Both noise processes $\mathcal{N}_1$ and $\mathcal{N}_2$ vary over time. $\mathcal{N}_1$ is damped to a desired standard deviation of the resulting action $\mathbf{u}$, while the amplitude of $\mathcal{N}_2$ is given as a function of an Ornstein–Uhlenbeck process. This different choice introduces two different time scales in the exploration process: when $\mathcal{N}_1$ is already damped, the noise process $\mathcal{N}_2$ still allows us to explore efficiently the control landscape in the vicinity of the converged policy. We refer to DDPG articles for further technical information about its implementation [41–43].

4. Control of the Kuramoto–Sivashinsky model

The DDPG algorithm introduced in the previous section is tested on the one-dimensional KS equation. The one-dimensional KS equation is used for the description of flame fronts and reaction–diffusion systems. It is well known and referenced as being one of the simplest nonlinear PDEs exhibiting spatio-temporal chaos [23,26], thus providing an appropriate test-bed for the proposed control strategy. In the following, we first describe the dynamics of the KS in the phase space (§4a), before a description of the controlled cases is given in §4b. A summary of the strategy and of the achieved results is shown in the electronic supplementary material.

(a) Dynamics of the Kuramoto–Sivashinsky equation

The time evolution of the velocity $v = v(x, t)$ on a periodic domain of length L is given by

$$\frac{\partial v}{\partial t} + v\frac{\partial v}{\partial x} = -\frac{\partial^2 v}{\partial x^2} - \frac{\partial^4 v}{\partial x^4} + \phi, \tag{4.1}$$

where ϕ is a spatio-temporal forcing term. The equation is characterized on the left-hand side by a nonlinear convective term and on the right-hand side by a diffusion term expressed by two addends: a second-order derivative related to energy production and a fourth-order derivative acting as a hyper-diffusion.

The length of the domain dictates different regimes for the solution. Introducing the trivial solution $E_0 = 0$, it can be shown that, for $L < L_c = 2\pi$, the dynamics is stable and converges towards E_0, while for $L > L_c$ a chaotic dynamics emerges (in a Lyapunov sense). In this work, we focus on the dynamics of the KS equation for a domain length of $L = 22$, identical to that studied in [26]. For this value of L, the KS equation exhibits some of the features found in Navier–Stokes when dissipation is high and the system spatial extent is small. In this case,

KS exhibits low-dimensional dynamical behaviours. This regime is often referred to as weak turbulence or, more properly, spatio-temporal chaos. The chaotic dynamics can be characterized by analysing the manifold associated with the invariant solutions [26,44]. From a practical point of view, this scenario is associated with transition-to-turbulence and self-sustaining mechanisms occurring in boundary layer flows [45]. From the control viewpoint, it is possible to control the KS system by linear controllers, as shown in [46]; yet, our aim here is to consider a fully data-driven RL strategy and show that it constitutes a feasible alternative to standard techniques of control.

(i) Numerical simulations

Numerical simulations are used for solving (4.1) with a code adapted from `pyks`.[2] As already mentioned, periodic boundary conditions are imposed, $v(x,t) = v(x+L,t)$. The spatial periodicity allows us to project the instantaneous solution onto Fourier modes and to perform spatial derivatives in the Fourier space. For $L = 22$, $N = 64$ Fourier collocation points have been used. This number of collocation points is deemed sufficient for an accurate representation of the spatio-temporal dynamics (see the related discussion in [26]). Time marching is carried out with a third-order semi-implicit Runge–Kutta scheme [47]. The linear operator on the right-hand side of (4.1) is marched in time by an implicit scheme, whereas the nonlinear and forcing terms are marched in time explicitly. This choice guarantees the stability of the numerical scheme at a reasonable computational cost [48]. For all numerical simulations, a time step of 0.05 was adopted.

In figure 3*a*, a sample of the chaotic dynamics is shown when the solution is initialized with the trivial state E_0 perturbed by a Gaussian white noise with an amplitude of order 10^{-4}. The trajectory evolves in time as shown in the corresponding phase space (figure 3*b*) obtained by projecting the dynamics onto the set of the dominant Fourier modes $\hat{e}_1$, $\hat{e}_2$, $\hat{e}_3$. Indeed, because of the periodicity of the domain, the velocity fields are characterized by different phases, with respect to which Fourier projection is independent. In figure 3*b*, the different invariant solutions computed by Newton iterations are shown, namely the three unstable, fixed points indicated by E_i ($i = 1,2,3$) and the two travelling waves indicated by TW_i ($i = 1,2$). Apart from the relevant role in the dynamics, the invariant solutions E_i are used as a target for our control strategy in §4b. A detailed analysis of the resulting dynamics shown in figure 3*a*,*b* is beyond the scope of the present paper and is already well described in [26]. However, it is interesting to observe how the system evolves by frequently visiting the vicinity of the solutions E_1 and E_2, as well as the two travelling waves; on the other hand, the dynamics is 'attracted' by the unstable solution E_3, along the manifold associated with it, without ever reaching it.

(b) Controlled system

As mentioned in §2, the plant is the system to be controlled, including the inputs (actuators) and the outputs (sensors) of the system. The number of actuators and sensors, their distribution and the combination of the two define a rather large parametric space. In this work, we consider four actuators equispaced along x. The support of each actuator is assumed to be Gaussian shaped and the forcing term in (4.1) is given by

$$\phi(x,t) = \sum_{i=1}^{4} u_i(t)(2\pi\sigma)^{-1/2} \exp\left(-\frac{(x - x_i^{\mathrm{a}})^2}{2\sigma^2}\right), \tag{4.2}$$

where $x_i^{\mathrm{a}} \in \{0,1,2,3\}L/4$ is the location of the actuators along the x-axis, $\sigma = 0.4$ defines their spatial distribution and $u_i(t)$ is the amplitude of the ith component of the action vector. Regarding the sensors, we make the realistic assumption that real-life controllers rely only on partial information. We then introduce eight sensors measuring the local velocity v; the sensors are

[2]See https://github.com/jswhit/pyks/blob/master/KS.py.

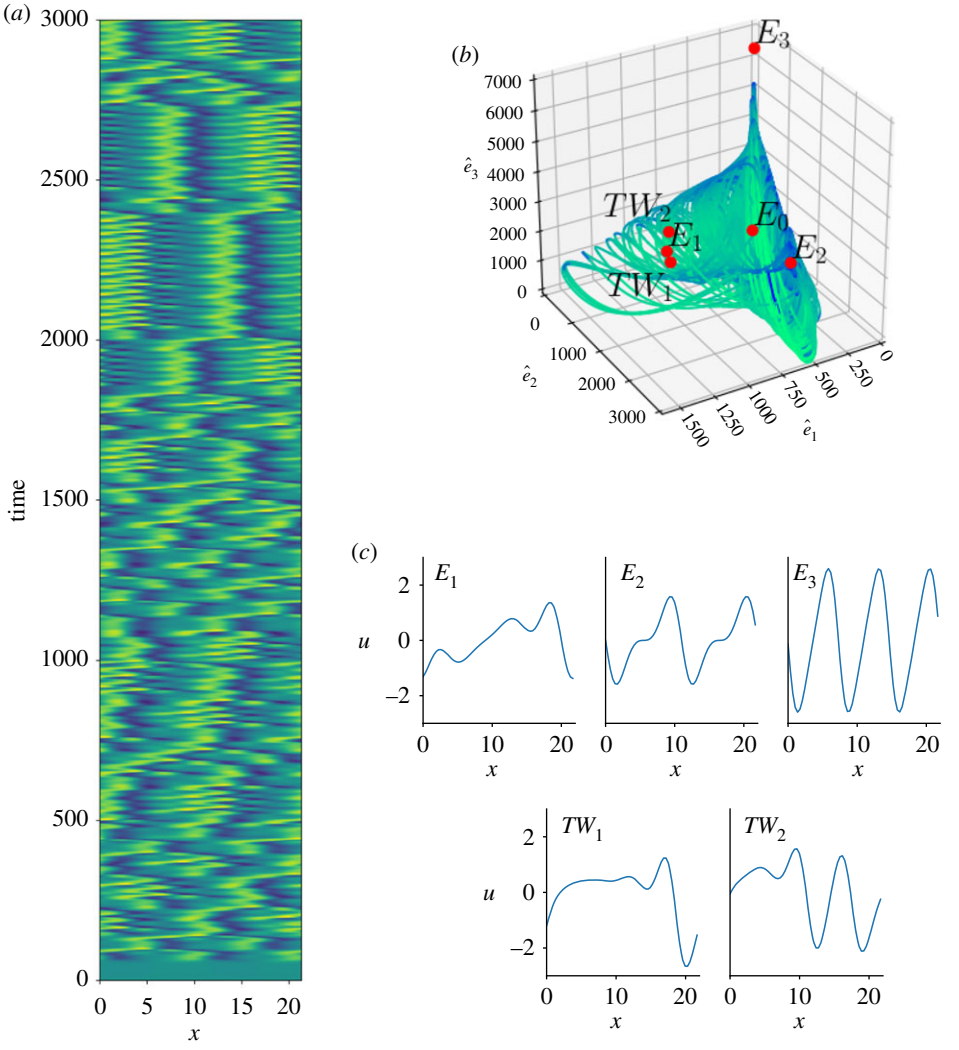

Figure 3. Dynamics of the KS equation on a time interval of 3000 time units. The contour plot in (*a*) shows the space–time behaviour of a trajectory emanating from the perturbed state E_0. The same trajectory is shown in (*b*) in the phase space spanned by the dominant Fourier coefficients $\{\hat{e}_1, \hat{e}_2, \hat{e}_3\}$. Red spots indicate: the trivial solution E_0, the three non-trivial invariant solutions $E_i, i = 1, 2, 3$, and the two travelling waves TW_1 and TW_2. The spatial distributions of the solutions are given in (*c*). Note how the trajectory is attracted by E_3, although never visited, while the other two equilibria are often visited. (Online version in colour.)

equispaced by $\Delta x = 2L/16$ and staggered with respect to the actuators' locations such that $x_i^s \in \{1, 3, 5, 7, 9, 11, 13, 15\}L/16$.

(i) Implementation of the control policies

The DDPG approach (see §3b) is implemented using scripts in `PyTorch`[3] and adapted from `PyTorch-ActorCriticRL`.[4] The critic and actor parts are both implemented by means of an NN. The NN of the critic part consists of an input layer of dimension 12, with four nodes dedicated to the actuators' signals and eight to the sensors' ones, and a single scalar output

[3]See https://pytorch.org/.

[4]See https://github.com/vy007vikas/PyTorch-ActorCriticRL.

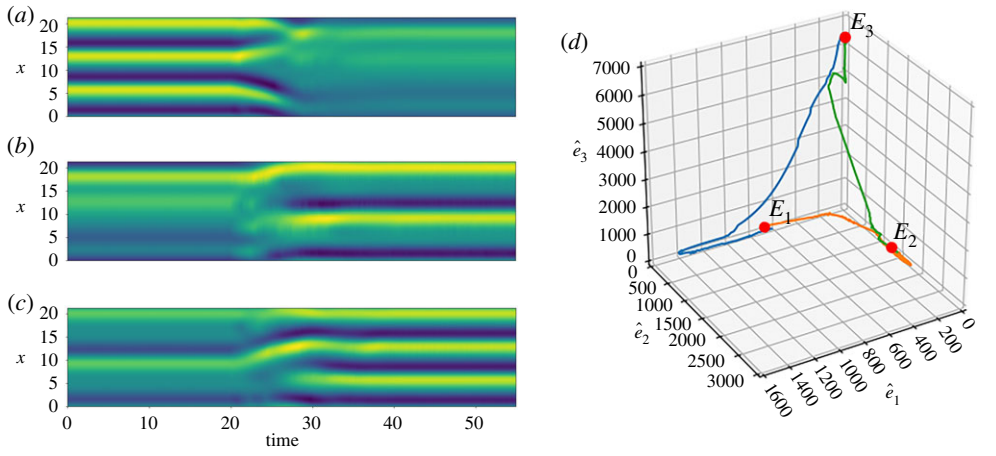

Figure 4. Closed-loop dynamics for the three control test cases: (*a*) $E_3 \to E_1$, (*b*) $E_1 \to E_2$, and (*c*) $E_2 \to E_3$. The controller is switched on at $t = 20$. The control strategy used in each test case is the optimal RL policy. The trajectories are shown in the Fourier phase space in (*d*). (Online version in colour.)

layer giving the reward; two hidden layers are introduced, with 256 and 128 nodes, respectively, both featuring the `swish` activation function. The NN for the actor part approximates the action vector. As such, the input layer is fed with the sensor measurements and is of dimension 8, while the output layer provides the control signal feeding the four actuators. Two hidden layers are used, of dimensions 128 and 64, with activation functions `swish` and `tanh`, respectively. The last layer acts as a saturating function; the maximum amplitude of the output is $\phi_{\max} = 0.5$. Because of the large number of hyper-parameters involved in the implementation of NNs, a few remarks are in order. First of all, we observed that the performance of the DDPG policies were essentially unchanged with respect to the number of hidden layers and the number of hidden neurons. Our choices were then mainly dictated by the compromise between computational costs during the learning process of the policies and the control performance. Concerning the activation functions, our preliminary tests—not included here for brevity—showed that the `ReLu` functions led to satisfactory results, but the performance in terms of convergence was generally lower than with the `swish` functions. This behaviour has been recently observed [49] for applications of deep networks to image classification and machine translation. The superiority of the `swish` functions over the `ReLu` functions was explained by the smoothness and non-monotonicity of the `swish` function around $x = 0$. Finally, the maximal amplitude of the output was chosen arbitrarily, mimicking a limitation often found in realistic applications. The training is performed using the `Adam` optimization algorithm, with a learning rate of 0.001 for both networks and a mini-batch with 200 examples of the optimization of the critic network. Finally, the Euclidean distance between the state of the system v and the non-trivial invariant solutions, namely E_1, E_2 and E_3 (see figure 3*c*), is chosen as the definition of the reward r to be minimized,

$$r := \|E_i - v\|_2. \tag{4.3}$$

Note that three different policies, one for each of the non-trivial target states E_i, are trained. The discount factor is set to $\gamma = 0.99$. In the following, for simplicity, we label the different policies with the invariant solution considered in their objective function.

(ii) Results

We want to demonstrate the ability of the DDPG-based policies to drive the chaotic system towards the unstable fixed points and keep the dynamics in their vicinity, using the localized sensors and actuators discussed before. From the engineering viewpoint, the KS system serves

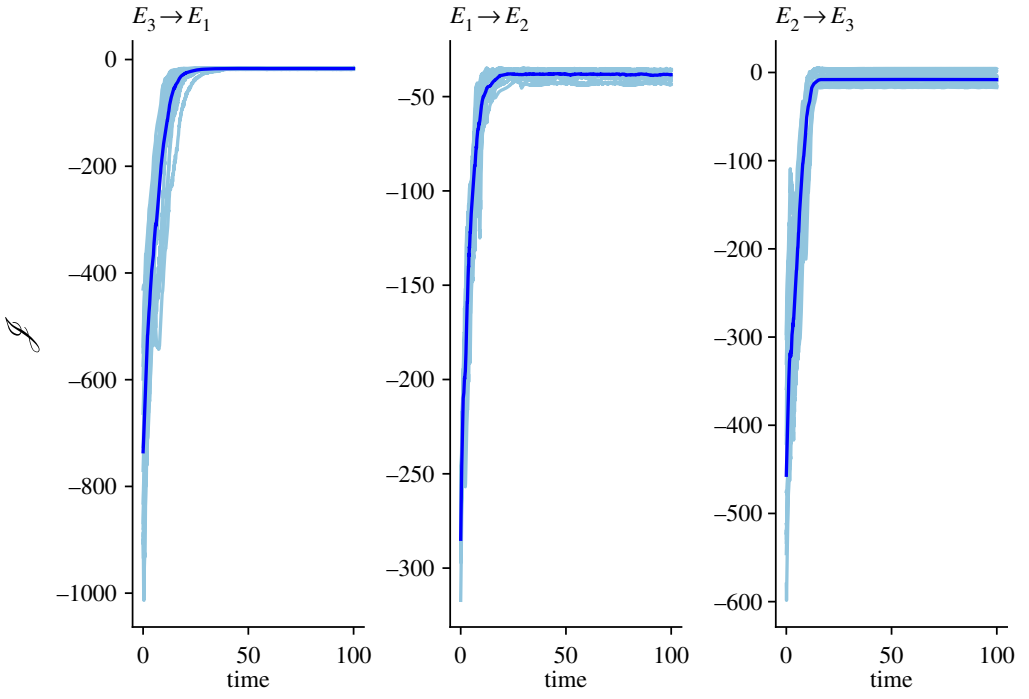

Figure 5. The value function is plotted for each of the three control test cases considered in figure 4. For each of the test cases, the robustness of the policy is assessed by running 15 different experiments emanating from different initial conditions of the control. The darker blue line indicates the average of the value as a function of time. For each case, it can be observed that—regardless of the initial conditions—the controller is capable of reaching the target in a robust manner. (Online version in colour.)

as a test-bed for the DDPG algorithm. Indeed, a realistic application of control to a fluid system can only rely on some measurements of the environment to drive the flow to a given operating condition.

First, we consider each individual policy for driving the dynamics of the system from one invariant solution to another. In figure 4, the spatio-temporal behaviour of the solution is shown in the inserts (*a*–*c*), and as a phase-space representation in (*d*). In each case, the RL controller is active for $t > 20$. For each control case, the numerical experiments are repeated 15 times with different initial conditions for the control to assess the robustness of the policy. In figure 5, we report the time evolution of the value function for each experiment (light blue). The darker line indicates the average of the runs. By comparing figures 4*d* and 5, we can observe that the controller is capable of driving the system in about 10 time units towards the target solution. We also show that the control action of the policies is robust since the general behaviour is comparable for all the runs considered.

To further verify the robustness of the policies, we consider a second set of numerical experiments, where the initial condition is randomly chosen in the phase space. This is shown in figure 6, where we consider 10 different initial conditions randomly and run the simulation under the control policy π_{E_3}. Five of these examples are reported in the inserts (*a*–*e*). The controller is capable of controlling the system in about 15 time units in all but one case (*c*), where it took about 40 time units. The phase-space view is shown in (*f*), where the trajectories in (*a*–*e*) are shown in colours, while the others are in grey. The time evolution of the corresponding value functions is shown in (*g*). Large excursions of $\mathscr{J}$ roughly correspond to fluctuations in the phase space, although all trajectories finally converge towards E_3. In principle, this behaviour is a consequence of the Markovianity of the underlying model of observables. Indeed, the Markovianity implies

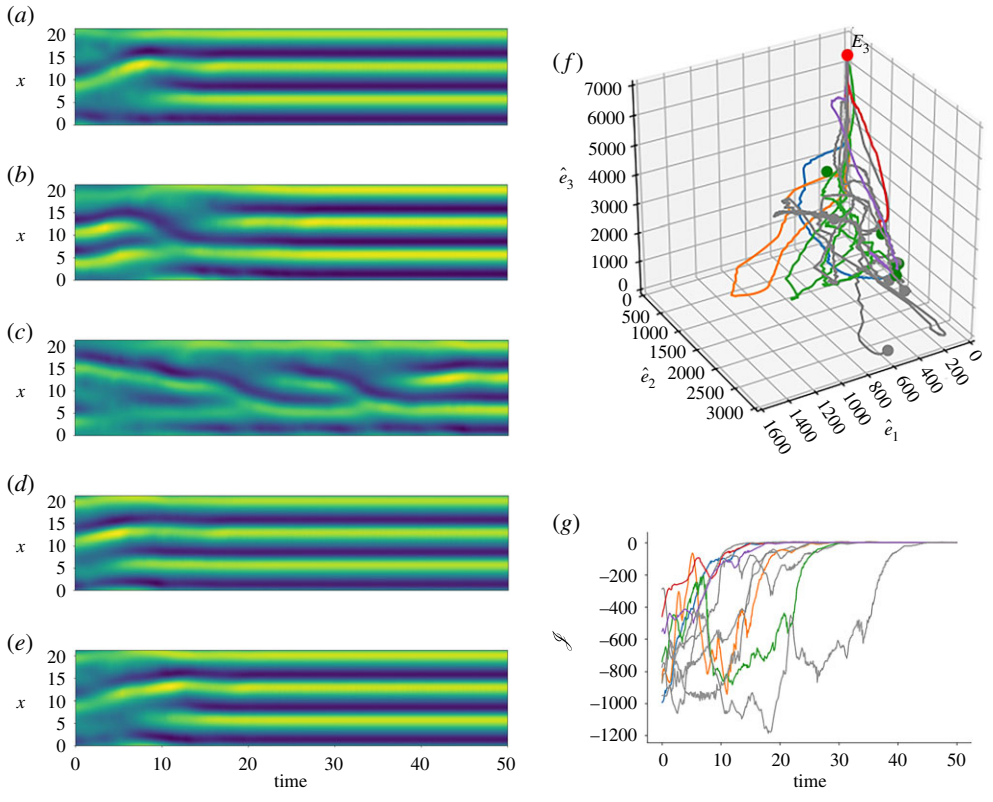

Figure 6. Illustration of the robustness of the closed-loop dynamics with respect to changes in the initial conditions. Ten different initial conditions are randomly chosen for the optimal policy driving the dynamics towards the unstable state E_3. Five of these examples are reported in (*a–e*). It is shown that, except for the trajectory shown in (*c*), the controller is capable of controlling the system in approximately 15 time units. The corresponding phase space is shown in (*f*), where the trajectories in (*a–e*) are shown in colours, while the others are in grey. The behaviour in time of the value functions associated with each of the simulations in (*f*) is shown in (*g*). (Online version in colour.)

that the model is memory-less and, within this limit, a critic-based algorithm is capable of driving the system towards the global minimum in accordance with the Bellman principle. In practice, the Markovianity cannot be guaranteed, although our results seem to corroborate this hypothesis because of the small impact of the initial conditions on the final performance. Regardless of the theoretical properties of the NN-based models, the independence from the initial conditions—as compared with linear control methods such as the model-based linear quadratic Gaussian—and the ability to maximize the value function demonstrate the level of performance and robustness of the DDPG-based strategy, when applied to the KS system. Similar robustness results exist for the control policies π_{E_1} and π_{E_2}.

5. Conclusion

We have presented a DRL algorithm successfully controlling the chaotic dynamics governed by the well-known nonlinear KS equation. The DRL combines RL principles with deep NNs for approximating the value function and the control policy. Among the different available strategies, we have tested an actor–critic algorithm, the DDPG. This choice is related to the possibility of directly tailoring the classical strategies from the nonlinear optimal control theory and, in particular, the solution of the HJB equation, with the resulting approximation obtained by DDPG. From this perspective, our approach departs from recent efforts found in the flow-control

literature as it allows knowledge of the cost landscape in the vicinity of the system trajectory, hence bringing robustness with respect to perturbations of the state in the phase space while allowing for high control performance. Further, our off-policy framework is not episode based, in the sense that it is not necessary to observe a trajectory (or a significant portion of it) before the critic and actor can improve, and it can then potentially require less training data (faster learning).

We emphasize that, in the present work, the controlled system exhibits a chaotic behaviour in the analysed regime. This configuration is significantly more involved than systems exhibiting a periodic, or quasi-periodic, dynamics. Nonetheless, we demonstrate that, using a limited, localized set of actuators and sensors and model-free DRL controllers, it is possible to drive and stabilize the dynamics of the KS system around its unstable fixed solutions, here considered as target states. Also, we show that the controllers are robust with respect to the initial conditions by considering numerous trajectories emanating from them; for all the tested cases, the performances are essentially unchanged with respect to the initial conditions.

The complexity of the KS system and the possibility of computing an RL control policy by solely relying on local measurements suggest the extension of this application to configurations of engineering interest, such as the control of drag/lift in turbulent boundary layers or in bluff bodies as well as the enhancement/reduction of turbulent mixing. In this sense, this work represents a first effort toward these applications. We are currently testing the DRL framework with canonical shear flows, in both numerical and experimental settings, by analysing the challenges that a real system poses in terms of limited and noisy observables, low measurement sampling frequency, high-dimensional state–action space and time delays. Future work will also be devoted to a more complete comparison with standard linear and nonlinear model-based strategies, as well as alternative algorithms from the DRL framework.

Data accessibility. The code used in this work is available at: https://flowcon.cnrs.fr/wp-content/uploads/2019/10/ddpg_KS.zip.

Authors' contributions. The project was initiated by L.M. and L.C. M.A.B. implemented all the routines used in this work for the analysis of the dynamics governed by the Kuramoto–Sivashinsky equation, the approximation and control by reinforcement learning, with supervision from O.S. and L.M. and feedback from A.A. The paper was written by M.A.B., O.S., L.M. and L.C., with feedback from G.W. and A.A.

Competing interests. We declare we have no competing interests.

Funding. This project was generously funded by the French Agence Nationale pour la Recherche (ANR) and Direction Générale de l'Armement (DGA) via the FlowCon project (no. PANR-17-ASTR-0022). L.C. was further partially supported by the COWAVE project (no. ANR-17-CE22-0008).

Acknowledgements. The authors are grateful to Sylvain Caillou for his support in the numerical implementation of the algorithm. This work has also benefited from discussions with Charles Pivot, who is gratefully acknowledged.

Appendix A. Solving the linear optimal problem: variational and HJB approaches

In this section we briefly summarize how a classic linear quadratic regulator (LQR) problem can be derived from the more general formulation of the HJB equation developed in §2b. This appendix is not meant to be exhaustive, but instead puts into perspective the application of RL with respect to standard approaches from control theory applied in the last decade in flow control. For more details on optimal control theory, the interested reader can refer to [30] and to previous reviews [1,2,50].

To begin with, we consider the assumption of a linear, time-invariant, state-space model

$$\frac{\mathrm{d}\mathbf{v}}{\mathrm{d}t} = A\mathbf{v} + B\mathbf{u} \tag{A 1a}$$

and

$$\mathbf{y} = C\mathbf{v} + D\mathbf{u}. \tag{A 1b}$$

In this case, the system matrix $A \in \mathbb{R}^{N\times N}$ is obtained from the discretization in space of the analysed model, including boundary conditions; the spatial distribution of the m actuators is

indicated by the matrix $B \in \mathbb{R}^{N\times m}$, while $C \in \mathbb{R}^{n\times N}$ is the matrix including n sensors and $D \in \mathbb{R}^{n\times m}$ is the feed-through (or feed-forward) matrix. The aim of LQR is to define a control signal $\mathbf{u}$, such that the quadratic objective function

$$\mathscr{J}(\mathbf{v}_t, \mathbf{u}_t) = \frac{1}{2}\int_0^T (\mathbf{v}^\mathsf{T} Q_1 \mathbf{v} + \mathbf{u}^\mathsf{T} Q_2 \mathbf{u})\,\mathrm{d}\tau \tag{A 2}$$

is minimized [30]. The first term of (A 2) is related to the energy of the state $\mathbf{v}$, while the second one penalizes the energy expense of the control action. In this definition, the matrices $Q_1 \succcurlyeq 0 \in \mathbb{R}^{N\times N}$ and $Q_2 \succ 0 \in \mathbb{R}^{m\times m}$, respectively, represent state and control penalties.

The objective is then to determine $\mathscr{J}^\star = \min_{\mathbf{u}} \mathscr{J}$ under the constraints of the state equation (A 1*a*). This optimal control problem can be solved by minimizing the augmented Lagrangian

$$\mathscr{L} = \frac{1}{2}\int_0^T (\mathbf{v}^\mathsf{T} Q_1 \mathbf{v} + \mathbf{u}^\mathsf{T} Q_2 \mathbf{u})\,\mathrm{d}\tau + \int_0^T \mathbf{p}^\mathsf{T}(\dot{\mathbf{v}} - A\mathbf{v} - B\mathbf{u})\,\mathrm{d}\tau, \tag{A 3}$$

where the governing equations with initial conditions $\mathbf{v} = \mathbf{v}_0$ act as a constraint and the *co-state* or *adjoint state* $\mathbf{p}$ can be interpreted as a Lagrangian multiplier. The *optimality system* is then obtained by imposing the optimality conditions on $\mathscr{L}$. It leads to the following system of coupled equations:

$$\mathscr{L}_{\mathbf{p}} = \mathbf{0} \implies \frac{\mathrm{d}\mathbf{v}}{\mathrm{d}t} = A\mathbf{v} + B\mathbf{u}, \qquad \mathbf{v}(0) = \mathbf{v}_0, \quad \text{Direct equation} \tag{A 4a}$$

$$\mathscr{L}_{\mathbf{v}} = \mathbf{0} \implies \frac{\mathrm{d}\mathbf{p}}{\mathrm{d}t} = -A^\mathsf{T}\mathbf{p} + Q_1\mathbf{v}, \quad \mathbf{p}(T) = \mathbf{0}, \quad \text{Adjoint equation} \tag{A 4b}$$

and

$$\mathscr{L}_{\mathbf{u}} = \mathbf{0} \implies \mathbf{0} = B^\mathsf{T}\mathbf{p} + Q_2\mathbf{u}. \quad \text{Optimality condition} \tag{A 4c}$$

The direct–adjoint system (A 4*a*) and (A 4*b*) is solved iteratively, by marching forward in time the direct equation, and backward in time the adjoint equation, forced by the second term on the right-hand side of (A 4*b*). The optimal condition is updated using the gradient given by (A 4*c*). It can be shown that the optimality system can be directly solved by means of an associated continuous time algebraic Riccati equation (CARE) [30].

We have shown in §3 that the HJB equation determines the minimum value function for a given dynamical system with an associated reward. This equation is directly linked to the class of optimal control problems treated previously. For this reason, we aim at obtaining the CARE starting from the HJB equation rewritten for the linear system as

$$-\dot{\mathscr{J}}^\star(\mathbf{v}, t) = \min_{\mathbf{u}} \mathscr{H}(\mathbf{v}(t), \mathbf{u}(t), \mathscr{J}^\star_{\mathbf{v}}, t) \tag{A 5}$$

$$= \min_{\mathbf{u}} \left[\frac{1}{2}(\mathbf{v}^\mathsf{T} Q_1 \mathbf{v} + \mathbf{u}^\mathsf{T} Q_2 \mathbf{u}) + \mathscr{J}^{\star\mathsf{T}}_{\mathbf{v}}(A\mathbf{v} + B\mathbf{u})\right], \tag{A 6}$$

where $\mathscr{H}$ by definition is the Hamiltonian. The right-hand side of (A 6) is minimized for $\mathscr{H}_{\mathbf{u}} = \mathbf{0}$, leading to the optimal control

$$\mathbf{u}^\star = -Q_2^{-1} B^\mathsf{T} \mathscr{J}^\star_{\mathbf{v}}, \tag{A 7}$$

where the existence of Q_2^{-1} is guaranteed, as Q_2 is a positive definite matrix. Plugging this expression in (A 6), we get

$$-\dot{\mathscr{J}}^\star = \tfrac{1}{2}\mathbf{v}^\mathsf{T} Q_1 \mathbf{v} - \tfrac{1}{2}\mathscr{J}^{\star\mathsf{T}}_{\mathbf{v}} B Q_2^{-1} B^\mathsf{T} \mathscr{J}^\star_{\mathbf{v}} + \mathscr{J}^{\star\mathsf{T}}_{\mathbf{v}} A\mathbf{v}. \tag{A 8}$$

At this point, the value function $\mathscr{J}^\star$ can be expressed as a function of the unknown, real symmetric positive-definite matrix $S(t)$

$$\mathscr{J}^\star(\mathbf{v}(t), t) = \tfrac{1}{2}\mathbf{v}^\mathsf{T}(t) S(t) \mathbf{v}(t). \tag{A 9}$$

This expression can be derived in time $\dot{\mathscr{J}}^\star = \frac{1}{2}\mathbf{v}^\mathsf{T}\dot{S}\mathbf{v}$, and with respect to the state $\mathbf{v}$, leading to the vector $\mathscr{J}^\star_{\mathbf{v}} = S\mathbf{v}$. Also, note that if we compare the optimal control (A 7) determined with the Hamiltonian and the one that can be deduced from the optimality condition (A 4*c*), we arrive

at $\mathbf{p} = \mathscr{J}^{\star}_{\mathbf{v}}$. Introducing this term in (A 8), and considering that only the symmetric part of $S(t)A$ contributes to the result, we obtain

$$-\mathbf{v}^{\mathsf{T}}\dot{S}(t)\mathbf{v} = \mathbf{v}^{\mathsf{T}}Q_1\mathbf{v} - \mathbf{v}^{\mathsf{T}}S(t)BQ_2^{-1}B^{\mathsf{T}}S(t)\mathbf{v} + \mathbf{v}^{\mathsf{T}}(S(t)A + A^{\mathsf{T}}S(t))\mathbf{v}, \tag{A 10}$$

from which it is possible to obtain the differential Riccati equation since the expression above is valid $\forall \mathbf{v}$,

$$-\dot{S}(t) = S(t)A + A^{\mathsf{T}}S(t) - S(t)BQ_2^{-1}B^{\mathsf{T}}S(t) + Q_1. \tag{A 11}$$

The optimal action is finally obtained as

$$\mathbf{u}^{\star} = -Q_2^{-1}B^{\mathsf{T}}S\mathbf{v} =: K\mathbf{v}, \tag{A 12}$$

where $K(t)$ is the optimal control gain obtained as a solution to the LQR problem. The corresponding time-invariant optimal control gain is obtained for $T \to \infty$, leading to the CARE. In perfect analogy, it is possible to define an optimal estimation problem that, using the LQR and owing to the separation principle, leads to the linear quadratic Gaussian compensator [30,31].

As a conclusion, some observations can be made. First of all, since the Riccati equation is not directly solvable for systems characterized by a large number of degrees of freedom, say $N > 10^4$, the direct–adjoint system (A 4*a*)–(A 4*b*) is often solved instead; see for instance [51] and [52]. Optimization based on a finite sliding temporal window leads to the class of model predictive controllers, often characterized by a nonlinear loop; examples in fluid mechanics are provided in a seminal work [53] and in other applications [54,55].

References

1. Kim J, Bewley TR. 2007 A linear systems approach to flow control. *Annu. Rev. Fluid Mech.* **39**, 383–417. (doi:10.1146/annurev.fluid.39.050905.110153)
2. Sipp D, Schmid PJ. 2016 Linear closed-loop control of fluid instabilities and noise-induced perturbations: a review of approaches and tools. *Appl. Mech. Rev.* **68**, 020801. (doi:10.1115/1.4033345)
3. Brunton S, Noack B. 2015 Closed-loop turbulence control: progress and challenges. *Appl. Mech. Rev.* **67**, 050801. (doi:10.1115/1.4031175)
4. Becker R, King R, Petz R, Nitsche W. 2007 Adaptive closed-loop control on a high-lift configuration using extremum seeking. *AIAA J.* **45**, 1382–1392. (doi:10.2514/1.24941)
5. Kegerise MA, Cabell RH, Cattafesta LN. 2007 Real time feedback control of flow-induced cavity tones—part 1: fixed-gain control. *J. Sound Vib.* **307**, 906–923. (doi:10.1016/j.jsv.2007.07.063)
6. Kegerise MA, Cabell RH, Cattafesta LN. 2007 Real time feedback control of flow-induced cavity tones—part 2: adaptive control. *J. Sound Vib.* **307**, 924–940. (doi:10.1016/j.jsv.2007.07.062)
7. Ma Z, Ahuja S, Rowley CW. 2011 Reduced-order models for control of fluids using the eigensystem realization algorithm. *Theor. Comput. Fluid Dyn.* **25**, 233–247. (doi:10.1007/s00162-010-0184-8)
8. Ljung L. 1986 *System identification: theory for the user*. Upper Saddle River, NJ: Prentice-Hall.
9. Van Overschee P, De Moor B. 2012 *Subspace identification for linear systems: theory—implementation—applications*. Boston, MA: Springer Science & Business Media.
10. Gautier N, Aider JL, Duriez T, Noack BR, Segond M, Abel M. 2015 Closed-loop separation control using machine learning. *J. Fluid Mech.* **770**, 442–457. (doi:10.1017/jfm.2015.95)
11. Sutton RS, Barto AG. 1998 *Introduction to reinforcement learning*, vol. 135. Cambridge, MA: MIT Press.
12. Silver D *et al.* 2017 Mastering the game of Go without human knowledge. *Nature* **550**, 354–371. (doi:10.1038/nature24270)
13. Lawhead R, Gosavi A. 2019 A bounded actor–critic reinforcement learning algorithm applied to airline revenue management. *Eng. Appl. Artif. Intell.* **82**, 252–262. (doi:10.1016/j.engappai.2019.04.008)
14. Choset H, Lynch KM, Hutchinson S, Kantor GA, Burgard W, Kavraki LE, Thrun S. 2005 *Principles of robot motion: theory, algorithms, and implementations*. Cambridge, MA: MIT Press.

15. Bellman R. 1958 Dynamic programming and stochastic control processes. *Inf. Control* **1**, 228–239. (doi:10.1016/S0019-9958(58)80003-0)
16. Gorodetsky AA, Karaman S, Marzouk YM. 2015 Efficient high-dimensional stochastic optimal motion control using tensor-train decomposition. In *Proc. of Robotics: Science and Systems, Rome, Italy, 13–17 July 2015*. Robotics Science and Systems Foundation.
17. Gorodetsky AA, Karaman S, Marzouk YM. 2018 High-dimensional stochastic optimal control using continuous tensor decompositions. *Int. J. Robot. Res.* **37**, 340–377. (doi:10.1177/0278364917753994)
18. Guéniat F, Mathelin L, Hussaini M. 2016 A statistical learning strategy for closed-loop control of fluid flows. *Theor. Comput. Fluid Dyn.* **30**, 1–14. (doi:10.1007/s00162-016-0392-y)
19. Pivot C, Cordier L, Mathelin L, Guéniat F, Noack BR. 2017 A continuous reinforcement learning strategy for closed-loop control in fluid dynamics. In *Proc. 35th AIAA Applied Aerodynamics Conf., Denver, CO, 5–9 June 2017*, p. 3566. Reston, VA: AIAA.
20. Pivot C, Gorodetsky AA, Mathelin L, Cordier L. 2018 Toward control of weakly observed nonlinear dynamical systems using reinforcement learning. In *Proc. SIAM UQ 18, Garden Grove, CA, 16–19 April 2018*. Philadelphia, PA: SIAM.
21. Verma S, Novati G, Koumoutsakos P. 2018 Efficient collective swimming by harnessing vortices through deep reinforcement learning. *Proc. Natl Acad. Sci. USA* **115**, 5849–5854. (doi:10.1073/pnas.1800923115)
22. Rabault J, Kuchta M, Jensen A, Reglade U, Cerardi N. 2019 Artificial neural networks trained through deep reinforcement learning discover control strategies for active flow control. *J. Fluid Mech.* **865**, 281–302. (doi:10.1017/jfm.2019.62)
23. Holmes P, Lumley J, Berkooz G. 1996 *Turbulence coherent structures, dynamical systems and symmetry*. Cambridge, UK: Cambridge University Press.
24. Bohr T, Jensen MH, Paladin G, Vulpiani A. 2005 *Dynamical systems approach to turbulence*. Cambridge, UK: Cambridge University Press.
25. Jiménez J, Moin P. 1991 The minimal flow unit in near-wall turbulence. *J. Fluid Mech.* **225**, 213–240. (doi:10.1017/S0022112091002033)
26. Cvitanović P, Davidchack RL, Siminos E. 2010 On the state space geometry of the Kuramoto–Sivashinsky flow in a periodic domain. *SIAM J. Appl. Dyn. Syst.* **9**, 1–33. (doi:10.1137/070705623)
27. Goodfellow I, Bengio Y, Courville A. 2016 *Deep learning*. Cambridge, MA: MIT Press. See http://www.deeplearningbook.org.
28. Kirk DE. 1970 *Optimal control theory: an introduction*. Englewood Cliffs, NJ: Prentice Hall.
29. Recht B. 2018 A tour of reinforcement learning: the view from continuous control. *Annu. Rev. Control Robot. Auton. Syst.* **2**, 253–279. (doi:10.1146/annurev-control-053018-023825)
30. Lewis FL, Vrabie D, Syrmos VL. 2012 *Optimal control*. New York, NY: John Wiley & Sons.
31. Glad T, Ljung L. 2000 *Control theory*. London, UK: Taylor & Francis.
32. Skogestad S, Postlethwaite I. 2005 *Multivariable feedback control, analysis to design*, 2nd edn. New York, NY: Wiley.
33. Bertsekas DP. 1996 *Dynamic programming and optimal control*, vol. 1. Belmont, MA: Athena Scientific.
34. Stengel RF. 1994 *Optimal control and estimation*. New York, NY: Dover.
35. Bellman R. 1957 A Markovian decision process. *J. Math. Mech.* **6**, 679–684. (doi:10.1512/iumj.1957.6.56038)
36. Bellman R. 1961 *Adaptive control processes: a guided tour*. Princeton, NJ: Princeton University Press.
37. Ruder S. 2016 An overview of gradient descent optimization algorithms. (https://arxiv.org/abs/1609.04747)
38. Watkins CJ, Dayan P. 1992 Q-learning. *Mach. Learn.* **8**, 279–292. (doi:10.1007/BF00992698)
39. Mnih V *et al.* 2015 Human-level control through deep reinforcement learning. *Nature* **518**, 529–533. (doi:10.1038/nature14236)
40. Mnih V, Badia AP, Mirza M, Graves A, Lillicrap T, Harley T, Silver D, Kavukcuoglu K. 2016 Asynchronous methods for deep reinforcement learning. In *Proc. of the 33rd Int. Conf. on Machine Learning, New York, NY, 20–22 June 2016*, pp. 1928–1937. JMLR: W&CP, vol. 48.
41. Lillicrap TP, Hunt JJ, Pritzel A, Heess N, Erez T, Tassa Y, Silver D, Wierstra D. 2015 Continuous control with deep reinforcement learning. (https://arxiv.org/abs/1509.02971)
42. Silver D, Lever G, Heess N, Degris T, Wierstra D, Riedmiller M. 2014 Deterministic policy gradient algorithms. In *Proc. of the 31st Int. Conf. on Machine Learning, Beijing, China, 22–24 June 2014*. JMLR: W&CP, vol. 32.

43. Schaul T, Quan J, Antonoglou I, Silver D. 2015 Prioritized experience replay. (https://arxiv.org/abs/1511.05952)
44. Greene J, Kim JS. 1988 The steady states of the Kuramoto-Sivashinsky equation. *Physica D* **33**, 99–120. (doi:10.1016/S0167-2789(98)90013-6)
45. Khapko T, Kreilos T, Schlatter P, Duguet Y, Eckhardt B, Henningson DS. 2016 Edge states as mediators of bypass transition in boundary-layer flows. *J. Fluid Mech.* **801**, 1–10. (doi:10.1017/jfm.2016.434)
46. Armaou A, Christofides PD. 2000 Feedback control of the Kuramoto–Sivashinsky equation. *Physica D* **137**, 49–61. (doi:10.1016/S0167-2789(99)00175-X)
47. Kar SK. 2006 A semi-implicit Runge–Kutta time-difference scheme for the two-dimensional shallow-water equations. *Mon. Weather Rev.* **134**, 2916–2926. (doi:10.1175/MWR3214.1)
48. Ascher UM, Ruuth SJ, Wetton BTR. 1995 Implicit-explicit methods for time-dependent partial differential equations. *SIAM J. Numer. Anal.* **32**, 797–823. (doi:10.1137/0732037)
49. Ramachandran P, Zoph B, Le QV. 2017 Searching for activation functions. (https://arxiv.org/abs/1710.05941)
50. Fabbiane N, Semeraro O, Bagheri S, Henningson DS. 2014 Adaptive and model-based control theory applied to convectively unstable flows. *Appl. Mech. Rev.* **66**, 060801. (doi:10.1115/1.4027483)
51. Bewley T, Luchini P, Pralits JO. 2016 Methods for solution of large optimal control problems that bypass open-loop model reduction. *Meccanica* **51**, 2997–3014. (doi:10.1007/s11012-016-0547-3)
52. Luchini P, Bottaro A. 2014 Adjoint equations in stability analysis. *Annu. Rev. Fluid Mech.* **46**, 493–517. (doi:10.1146/annurev-fluid-010313-141253)
53. Bewley TR, Moin P, Temam R. 2001 DNS-based predictive control of turbulence: an optimal benchmark for feedback algorithms. *J. Fluid Mech.* **447**, 179–225. (doi:10.1017/S0022112001005821)
54. Cherubini S, Robinet JC, De Palma P. 2013 Nonlinear control of unsteady finite-amplitude perturbations in the Blasius boundary-layer flow. *J. Fluid Mech.* **737**, 440–465. (doi:10.1017/jfm.2013.576)
55. Xiao D, Papadakis G. 2017 Nonlinear optimal control of bypass transition in a boundary layer flow. *Phys. Fluids* **29**, 054103. (doi:10.1063/1.4983354)